

Функції

Процедурне та функціональне програмування

Одним з основних і найочевидніших недоліків програм, написаних у імперативному стилі програмування (послідовність операцій введення-виведення, інструкцій присвоєння, умовних інструкцій та циклів), є дублювання коду. Імперативний стиль зручний лише для невеликих програм. Для більшості прикладних задач він неефективний. Йому на зміну прийшло процедурне програмування, в основі якого лежить поняття функції як засобу групування інструкцій з метою їх подальшого використання. Функції дають програмістам дві основні переваги:

- 1) позбавляють необхідності багаторазово повторювати одні й ті самі фрагменти коду, полегшують коригування програм (зміна вноситься один раз у функції, а не кожного разу, де вона використовується);
 - 2) дають змогу розкласти складну задачу на прості частини (декомпозиція).
-

Створення і виклик функції

Функції прийнято іменувати з малої літери, додаючи між словами-складовими імені символ підкреслення (змійний регістр). Для створення (оголошення) функції використовується інструкція `def`:

```
def <ім'я_функції>(<ім'я_аргумента1>, <ім'я_аргумента2>, ...):  
    # тіло функції  
    <інструкція 1>  
    <інструкція 2>  
    ...
```

Тіло функції починає виконуватися в момент виклику функції, синтаксис якого:

```
<ім'я_функції>(<значення_аргумента1>, <значення_аргумента2>, ...)
```

Викликати функцію можна з будь-якого місця програми (у тому числі й з тіла іншої або навіть тієї ж самої функції¹), але після інструкції `def`, якою ця функція була оголошена.

Якщо функція повертає якісь значення, то вони повинні вказуватись в інструкції `return` у тілі функції. Якщо функція виконує певні операції над об'єктами,

¹ Такі функції називаються рекурсивними

але не повертає ніяких значень (в інших мовах програмування такі функції зазвичай називаються процедурами), то інструкція `return` може бути відсутня.

Також прийнято тіло функції починати багаторядковим коментарем (документацією), у якому лаконічно інформується про те, для чого призначена (що робить) ця функція. Для доступу до такої документації функції використовується синтаксис

```
<ім'я_функції>.__doc__
```

Приклад:

```
def some_function(x):  
    return len(str(x))
```

Результат виконання цієї функції можна записати у змінну:

```
y = some_function(100)
```

або вивести безпосередньо:

```
print(some_function(100))      # 3
```

Функція може не мати жодного аргумента:

```
def nearby_zero():  
    return 1E-10
```

Інструкція `return`

Під час виклику функції виконання основної програми призупиняється, а керування передається цій функції. Після завершення роботи функція за допомогою інструкції `return` передає результуючі об'єкти назад програмі, яка її викликала.

Якщо інструкція `return` відсутня, то функція повертає значення `None` (фактично в кінці функції Python неявно додає `return None`).

Наприклад, функція

```
def greeting():  
    print('Hello!')
```

просто виводить на екран текст привітання, не передаючи нічого (тобто, передаючи `None`) основній програмі. Перевірити, що функція дійсно повертає `None`, можна так:

```
print(greeting())
```

Крім виконаного тіла функції (друку привітання), буде виведено None.

Інструкція `return` може міститися у будь-якому місці тіла функції. Також у тілі функції можуть міститися кілька інструкцій `return`. Як тільки виконається одна з них, функція припиняє роботу і повертає результат програмі, яка її викликала. Наприклад, наведена нижче функція перевіряє, чи переданий їй список порожній:

```
def is_empty(lst):  
    if lst:  
        return False  
    else:  
        return True
```

інструкція `else` є зайвою, оскільки перехід до другої за ліком інструкції `return` відбудеться, лише якщо умова в інструкції `if` не виконається, що відповідає логіці програми. Тобто, умовні інструкції всередині функцій зазвичай записують так:

```
def is_empty(lst):  
    if lst:  
        return False  
    return True
```

Функція може повертати більше одного значення. Ці значення вказуються через кому після інструкції `return`:

```
def solve_abs_equation(k):  
    """Solution of equation |x| = k"""  
    if k == 0:  
        return 0  
    if k > 0:  
        return -k, k
```

Щоправда, ці значення все одно будуть “запаковані” в один об’єкт — кортеж:

```
print(solve_abs_equation(5))  # (-5, 5)
```

Для того, щоб отримати конкретні значення, потрібно звернутися до потрібного елемента кортежа за його індексом

```
neg_sol = solve_abs_equation(5)[0]
```

або використати групове присвоєння:

```
neg_sol, pos_sol = solve_abs_equation(5)
```

Важливо пам'ятати, що змінювані типи передаються функції за посиланням:

```
def swap(lst, i, j):  
    lst[i], lst[j] = lst[j], lst[i]  
  
lst = [1, 2, 3]  
swap(lst, 1, 2)  
print(lst)      # [1, 3, 2]
```

Області видимості, локальні та глобальні змінні

Кожного разу, зустрічаючи в коді програми ім'я (змінної, функції тощо), інтерпретатор шукає або створює його в просторі імен. Кожне ім'я має свою область видимості — місце у програмному коді, де цьому імені було присвоєно певне значення.

Будь-які змінні, яким були присвоєні значення всередині функції, включаючи аргументи функції, з'являються в момент виклику функції і зникають, коли функція завершує роботу і передає керування програмі, яка її викликала. Через цю свою особливість вони називаються локальними змінними. Тобто, кожна функція створює свій локальний простір імен, до якого не можна звернутися ззовні, а кожен виклик функції створює нову локальну область видимості.

Змінні, яким присвоєні значення поза усіма інструкціями `def`, називаються глобальними². Глобальна область видимості охоплює єдиний файл (модуль), у якому збережено код. Якщо потрібно використовувати імена з іншого модуля, його потрібно явно імпортувати.

Якщо всередині функції використовується якесь ім'я, інтерпретатор спочатку шукає його в локальній області видимості інструкції `def`. Не знайшовши цього імені, він здійснює пошук в локальних областях видимості охоплюючих інструкцій `def` (якщо такі є), потім у глобальній області видимості (в межах файлу, включаючи безпосередньо імпортовані модулі), і наостанок у вбудованій області видимості (вбудовані винятки та функції). Якщо в жодній з областей видимості імені не буде знайдено, з'явиться повідомлення про помилку.

Наприклад, у коді

```
def hi():  
    print(f"Hi, {name}")  
  
name = 'Alice'  
hi()
```

² Розрізняють також нелокальні (оголошуються інструкцією `nonlocal`) змінні - вони виникають у випадку вкладених інструкцій `def`. Нелокальні змінні виносяться на самостійне опрацювання для бажаних

інтерпретатор, не знайшовши значення змінної `name` в локальній області видимості (всередині функції `hi()`), підставляє значення глобальної (визначеної поза інструкцією `def`) змінної з таким самим іменем.

Локальні імена не конфліктують з зовнішніми іменами, навіть якщо вони збігаються:

```
def func():
    x = 'local'

x = 'global'
func()
print(x)          # global
```

У наведеному вище фрагменті коду, не зважаючи на те, що виклик функції змінив значення локальної змінної з іменем `x`, значення однойменної глобальної змінної не змінилося. Це пояснюється тим, що дві змінні з однаковим іменем у цьому випадку мають різні області видимості.

Вище було наголошено на тому, що аргументи змінюваних типів передаються функції за посиланням (див. приклад використання функції `swap()`). Аналогічно, якщо функція не знайде локального об'єкту змінюваного типу, то вона змінюватиме глобальний об'єкт з таким самим іменем (звісно, якщо такий об'єкт існує):

```
def add_to_list(item):
    lst.append(item)

lst = [1, 2, 3, 4, 5]
add_to_list(6)
print(lst)          # [1, 2, 3, 4, 5, 6]
```

Щоб дати змогу функції змінювати глобальну змінну незмінюваного типу (число, рядок, кортеж тощо), ім'я цієї змінної оголошується всередині функції інструкцією `global`:

```
def rename(new_name):
    global name
    name = new_name

name = 'Alice'
rename('Bob')
print(name)          # Bob
```

Позиційні та іменовані аргументи

За замовчуванням співставлення аргументів здійснюється згідно з їхніми позиціями зліва направо. Під час виклику функції їй потрібно передати рівно стільки аргументів, скільки імен аргументів вказано в заголовку функції, зберігаючи порядок. Якщо функції передати іншу (більшу або меншу) кількість аргументів, то буде згенеровано повідомлення про помилку:

```
from time import sleep

def timer(seconds, step):
    """Зворотний відлік часу
       від seconds секунд з заданим кроком step"""
    for second in range(seconds, 0, -step):
        print(second, end = '-')
        sleep(step)
    print('stop')

timer(10,1)      # 10-9-8-7-6-5-4-3-2-1-stop
timer(10)
# TypeError: timer() missing 1 required positional argument: 'step'
timer(10,1,2)
# TypeError: timer() takes 2 positional arguments but 3 were given
```

Іменовані аргументи дають змогу співставити аргументи за їхніми іменами. Порядок аргументів під час виклику функції у цьому випадку не має значення:

```
timer(step=2, seconds=10)          # 10-8-6-4-2-stop
```

Можна змішувати позиційні та іменовані аргументи. Тоді спочатку будуть співставлені зліва направо всі позиційні, а потім іменовані.

Python дає багато гнучких можливостей передавання іменованих аргументів, з якими можна ознайомитися у документації. Наприклад, таке оголошення функції вказує на те, що крок має передаватися виключно іменованим аргументом:

```
def timer(seconds, *, step):
    ...

timer(10, 2)
# TypeError: timer() takes 1 positional argument but 2 were given
timer(10, step=2)          # 10-8-6-4-2-stop
timer(step=2, seconds=10)  # 10-8-6-4-2-stop
```

Значення за замовчуванням

Існує можливість задавати значення аргументів за замовчуванням, які використовуватимуться, якщо під час виклику функції цим аргументам не передаватиметься конкретне значення:

```
def timer(seconds, step=1):
    for second in range(seconds, 0, -step):
        print(second, end = '-')
        sleep(step)
    print('stop')

timer(10)          # 10-9-8-7-6-5-4-3-2-1-stop
timer(10, 2)       # 10-8-6-4-2-stop
```

У наведеному вище коді аргументу `step` оператором присвоєння встановлене значення за замовчуванням 1. Під час першого виклику функції значення другого аргумента не вказано, тому інтерпретатор призначив йому значення 1. Під час другого виклику аргумент `step` прийняв передане йому значення 2, проігнорувавши значення за замовчуванням.

Передавання довільної кількості аргументів

Функція може приймати довільну кількість аргументів за допомогою параметрів `*args` і `**kwargs`. Перший параметр збирає додаткові позиційні аргументи в кортеж `args`, другий збирає додаткові іменовані аргументи в словник `kwargs`. Насправді імена `args` і `kwargs` просто домовленість, вони можуть бути іншими. Синтаксис визначається лише “зірочками”. Принцип роботи цих параметрів стане зрозумілим з такого коду:

```
def print_friends(person, *friends):
    s = f"{person}'s friends:\n"
    for friend in friends:
        s += friend + ', '
    print(s[:-2] + '.')

print_friends('Alice', 'Bob', 'Eve', 'John')
# Alice's friends: Bob, Eve, John.

def print_info(**kwargs):
    for key in kwargs:
        print(f'{key}: {kwargs[key]}')
```

```
print_info(name = 'Alice', age = 20)
# name: Alice
# age: 20
```

Функція `print_friends()` прийняла значення `'Alice'` позиційного аргумента `person`, а решту значень позиційних аргументів зібрала в кортеж `('Bob', 'Eve', 'John')` з іменем `friends`, по елементах якого зробила потім обхід у циклі `for`.

Функція `print_info()` приймає довільну кількість іменованих аргументів (у нашому випадку 2), формує з них словник `{'name': 'Alice', 'age': 20}` з іменем `kwargs` і виводить його елементи на екран.

Оскільки параметри `*args` і `**kwargs` працюють з різними типами аргументів (позиційними та іменованими відповідно), то їх можна використовувати одночасно в межах однієї функції.

Розпаковування аргументів

Якщо `*` (`**`) розмістити перед ітерабельним об'єктом³ під час виклику функції, то елементи цього об'єкту будуть розпаковані та передані функції як позиційні (іменовані) аргументи, наприклад:

```
def print_friends(person, *friends):
    s = f'{person}'s friends:\n"
    for item in friends:
        s += item + '\n'
    print(s)
```

```
f = ['Bob', 'Kate', 'Peter']
print_friends('Alice', *f)
```

```
def person_info(name, year_of_birth):
    print(f'My name is {name}. I was born in {year_of_birth}.')
```

```
person = {'year_of_birth': 1976, 'name': 'Roman'}
person_info(**person)
# My name is Roman. I was born in 1976.
```

Використання `**` у випадку словників забезпечує передавання значень за відповідними ключами. Використання `*` теж спрацює, але передаватимуться не значення, а ключі.

³ Ітерабельний об'єкт – об'єкт, елементи якого можна перебирати по одному

Декомпозиція

Зазвичай функції використовуються для декомпозиції задачі. Тобто, задача розбивається на складові частини (підзадачі), для кожної з яких створюється своя функція. Часто функції описуються в окремому файлі (модулі) і імпортуються в основну програму. Це дає змогу багаторазово використовувати функції в різних програмах, імпортуючи їх з потрібного модуля.

Продемонструємо це на прикладі задачі, яка за даними початковими показами виводить покази годинника через заданий проміжок часу. Вважатимемо, що час вводиться і виводиться рядком формату 'години:хвилини.секунди'. Усі необхідні функції зберігатимемо у модулі (файлі) `funcs.py`.

Розкладемо задачу на складові:

- 1) введення початкового показу годинника та інтервалу часу (за це відповідатиме функція `input_time()`);
- 2) перетворення початкового показу годинника в загальну кількість секунд, які пройшли від початку доби (за це відповідатиме функція `time_to_sec()`);
- 3) перетворення проміжку часу в секунди (за це теж відповідатиме функція `time_to_sec()`);
- 4) збільшення початкового показу (у секундах) на проміжок часу (теж у секундах); цю звичайну операцію додавання можна теж вкласти (інкапсулювати) у функцію, але не робитимемо цього;
- 5) перетворення отриманої кількості секунд у формат часу (за це відповідатиме функція `sec_to_time()`).
- 6) друк отриманого часу.

Тоді файл основної програми матиме приблизно такий вигляд⁴:

```
from funcs import *
time = input_time('Введіть початкові покази годинника: ')
interval = input_time('Введіть інтервал часу: ')
initial_seconds = time_to_sec(time)
interval_seconds = time_to_sec(interval)
total_seconds = initial_seconds + interval_seconds
time = sec_to_time(total_seconds)
print(time)
```

У файлі `funcs.py` мають бути описані всі використані функції:

```
def input_time(text):
    time = input(text)
    colon_index, dot_index = time.find(':'), time.find('.')
    return [int(time[colon_index]),
            int(time[colon_index+1:dot_index]),
            int(time[dot_index+1:])]
```

⁴ Приклад демонстраційний. Звісно, простіше було би скористатися модулем стандартної бібліотеки `datetime`

```
def time_to_sec(time):  
    return time[0]*3600 + time[1]*60 + time[2]  
  
def sec_to_time(seconds):  
    seconds = seconds % 86400  
    hours = seconds // 3600  
    minutes = (seconds % 3600) // 60  
    seconds = (seconds % 3600) % 60  
    return f'{hours:02}:{minutes:02}:{seconds:02}'
```

Частина коду основної програми, яка відповідає за обробку вхідних даних та відображення результату, можна теж вкласти у функцію модуля funcs.py, наприклад:

```
def show_new_time(time, interval):  
    print(sec_to_time(time_to_sec(time) + time_to_sec(interval)))
```

Тоді остаточний вигляд програми матиме більш компактний і читабельний вигляд:

```
from funcs import *  
  
time = input_time('Введіть початкові покази годинника: ')  
interval = input_time('Введіть інтервал часу: ')  
show_new_time(time, interval)
```

Рекурсивні функції

Деякі задачі шляхом декомпозиції можна звести до підзадач, які розв'язуються за таким самим алгоритмом. Наприклад, щоб обчислити суму елементів списку, потрібно до першого елемента цього списку додати суму елементів списку, який утворений з попереднього викиданням першого елемента.

Оскільки алгоритми розв'язання основної задачі та її складових однакові, то для них може бути написана одна єдина функція. Така функція називається рекурсивною.

Особливість рекурсивної функції, порівняно з іншими, в тому, що всередині вона викликає саму себе. Для запобігання нескінченному спрацьовуванню тіло рекурсивної функції повинно містити принаймі одну умовну інструкцію, у якій перевіряється, чи потрібно продовжувати виконання функції. Наприклад, умовою закінчення сумування елементів списку може бути досягнення на якомусь етапі порожнього списку (списку з 0 елементів, сума яких, очевидно, 0), а сама функція матиме такий вигляд:

```
def list_sum(L):  
    if not L:  
        return 0  
    return L[0] + list_sum(L[1:])
```

Анотування функцій⁵

Анотації функцій документують типи аргументів і тип значення, яке повертає функція. Анотування здійснюється у заголовку функції. Типи аргумента вказується через двокрапку після імені цього аргумента, а тип значення, яке повертається, після символів “->” в кінці заголовку функції, але перед двокрапкою:

```
def digits(s: str) -> int:  
    """Обчислює кількість цифр у рядку"""  
    n = 0  
    for symbol in s:  
        if symbol.isdigit():  
            n += 1  
    return n  
  
print(digits('У 2018 р. населення України складало 40 млн.')) # 6
```

Анотація наведеної вище функції інформує про те, що їй передається рядок (тип str), а вона повертає ціле число (тип int).

Анотування функцій необов'язкове. Анотації описують особливості функції, але не впливають на виконання, так як інтерпретатор не перевіряє відповідність типів. Основне призначення анотацій — полегшити користувачам розуміння написаного коду.

Лямбда-функції

Лямбда-функції — це невеликі функції, які можуть повертати значення лише одного виразу і оголошуються ключовим словом `lambda`. Загальний синтаксис лямбда-функції:

```
lambda аргумент_1, аргумент_2, ..., аргумент_n: вираз
```

⁵ Анотувати можна не тільки заголовок функції, а й змінні у її тілі. Модуль `typing` надає розширені можливості анотування

Наприклад, лямбда-функцію для обчислення суми, яка буде на рахунку через years років при початковому внеску deposit та pc складних річних відсотках, можна оголосити та викликати так:

```
money = lambda deposit, years, pc: deposit * (1 + pc/100)**years
my_money = money(1000, 2, 10)
print(round(my_money, 2))
```

1210.0

Фактично це скорочена (в один рядок) форма оголошення функції

```
def money(deposit, years, pc):
    return deposit * (1 + pc/100)**years
```

Але є відмінність. Лямбда-функцію не обов'язково називати — вона може бути анонімною (через це лямбда-функції часто називають анонімними функціями) і викликатися у тому ж рядку, що й оголошується. Зазвичай вони використовуються під час відображення функцій на послідовності під час сортування та інших операцій над колекціями:

```
persons = [('Kate', 19), ('Alice', 20), ('Bob', 18)]
persons.sort(key=lambda x: x[-1])
print(persons) # [('Bob', 18), ('Kate', 19), ('Alice', 20)]
```

Оперування функціями як об'єктами

Функції нічим не відрізняються від інших об'єктів Python — вони мають атрибути, їх можна присвоювати змінним, зберігати в структурах даних, передавати аргументами, повертати як результат виконання інших функцій, оголошувати всередині інших функцій та ін. Для демонстрації сказаного оголосимо функцію new_price, яка застосовує до старої ціни old_price знижку discount:

```
def new_price(old_price, discount):
    return old_price * (1 - discount/100)
```

Присвоїмо ім'я функції новій змінній:

```
new_cost = new_price
```

Тепер наступні два рядки коду даватимуть ідентичні результати:

```
print(new_price(50, 5)) # 47.5
print(new_cost(50, 5)) # 47.5
```

При цьому другий виклик працюватиме навіть після вилучення першого імені функції, оскільки об'єкт функції та її ім'я не одне й те саме:

```
del new_price
print(new_price(50,5))
# NameError: name 'new_price' is not defined
print(new_cost(50,5))
# 47.5
```

Цікавим застосуванням може слугувати приклад створення консольного меню на основі словника, значеннями якого є імена функцій:

```
import sys

def foo1():
    print('do 1')

def foo2():
    print('do 2')

menu = {'1': foo1, '2': foo2, '3': sys.exit}
while True:
    menu[input('? ')]()
```

Функція може бути аргументом іншої функції, наприклад:

```
def new_price(func, price, percent):
    return func(price, percent)

print(new_price(new_cost, 50, 5))    # 47.5
```

Розглянемо наступний фрагмент коду:

```
def apply_discount(percent):
    def discount(price):
        return price * (1 - percent/100)
    return discount
```

По-перше, у ньому всередині функції `apply_discount` оголошено іншу – `discount`. По-друге, результатом виконання функції `apply_discount` є не якесь значення, а функція. Це дає змогу створити функції для конкретної (у відсотках) знижки і надалі передавати їм аргументом початкову ціну:

```
discount_5_percent = apply_discount(5)
discount_10_percent = apply_discount(10)
```

```
print(discount_5_percent(50))      # 47.5
print(discount_10_percent(50))     # 45.0
```

Оскільки ім'я вкладеної функції `discount` ніде не фігурує, то функцію `apply_discount` можна переписати простіше з використанням анонімної лямбда-функції:

```
def apply_discount(percent):
    return lambda price: price * (1 - percent/100)
```

Фактично функція `apply_discount` слугує фабрикою для створення інших функцій. Вони можуть бути збережені, наприклад, у списку, і викликатися у циклі:

```
for func in [discount_5_percent, discount_10_percent]:
    print(func(50))
```

Функціональне програмування

Самостійно розберіть наступні приклади, які демонструють використання функцій `map`, `zip`, `filter`.

```
names = ['Alice', 'Bob', 'Kate']
L = map(len, names)
print(L)  # <map object at 0x7f48c8547c10>
for item in L:
    print(item)
# 5
# 3
# 4
print(list(L))  # [5, 3, 4]
```

```
x, y, z = map(float, input('Coordinates: ').split())
print(x, y + z)
Coordinates: 1 2 3
1.0 5.0
```

```
x = [1, 2, 3, 4]
y = ['one', 'two', 'three']
for item in zip(x, y):
    print(item)
# (1, 'one')
# (2, 'two')
# (3, 'three')
```

```
matrix = [[1, 2, 3], [4, 5, 6]]
m = list(map(list, list(zip(*matrix))))
print(m)  # [1, 4], [2, 5], [3, 6]
```

```
numbers = [1, 4, 8, 5, 12]
evens = list(filter(lambda x: not x%2, numbers))
print(evens)  # [4, 8, 12]
for item in filter(lambda x: x%2 == 0, numbers):
    print(item)
# 4
# 8
# 12
```